

MPI-Style Workflows

July 28: Simulated Distributed Systems

Ranks • collectives • scatter/gather/reduce • mini-MPI



MPI

Big idea

Distributed computing is about explicit communication among workers.



Student output

Implement mini-MPI primitives using multiprocessing.

Today's Roadmap

From process pools to MPI-style communication

01

Recap

shards and
parallel
inference

02

MPI model

rank, world size,
root,
communicator

03

Collectives

broadcast,
scatter,
gather,
reduce

04

Simulation

queues,
workers,
messages,
barriers

05

Lab

build mini-
MPI and
distributed
word count

1. Recap and Motivation

Why simulate distributed systems before using real clusters

Day 4 Recap: Shards Become Distributed Work

Inference taught split → compute → gather

S

Split

divide data into chunks or requests

C

Compute

workers run local work independently

A

Aggregate

combine predictions, metrics, or partial counts

Day 5 adds explicit communication patterns: who sends what to whom?

Why Learn MPI-Style Thinking?

These ideas appear in HPC, cloud systems, and distributed AI

SIM

Weather simulation

domain decomposition +
reductions

WEB

Search indexing

scatter documents, gather
indexes

REC

Recommendation

score user shards, reduce top-k

LLM

LLM serving

route requests, aggregate
telemetry

DL

Distributed training

broadcast parameters, all-reduce
gradients

ETL

Data analytics

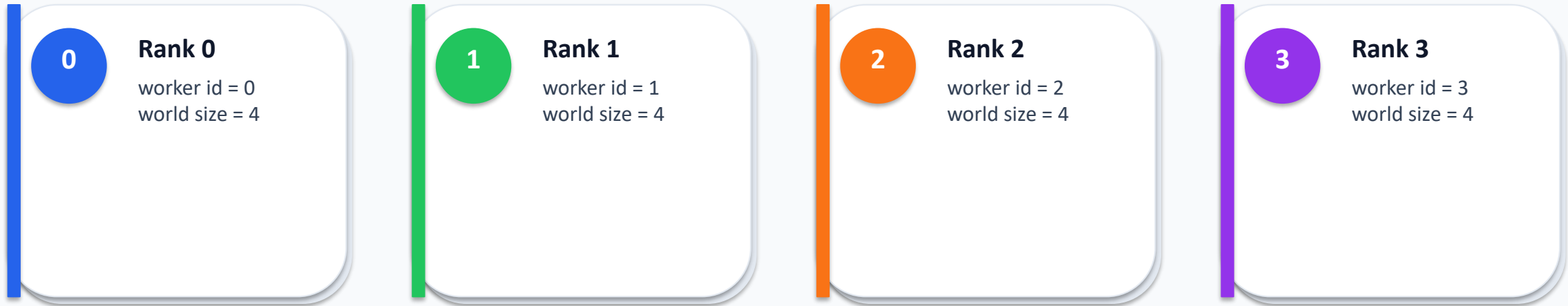
map chunks, reduce summaries

2. MPI Mental Model

Rank, world size, root, communicator, messages

MPI Mental Model

Many processes run the same program but know their rank



Rank = process identity; world size = number of participating processes

Key Terms

Vocabulary for collective communication

R

Rank

unique integer ID for a process

N

World size

number of processes in the group

0

Root

special source or destination rank

C

Communicator

group/context of participating processes

msg

Message

data exchanged between processes

all

Collective

operation involving all processes in a group

SPMD: Same Program, Multiple Data

Each rank branches based on its identity



```
# Pseudocode for SPMD style
rank = get_rank()
world_size = get_world_size()

if rank == 0:
    data = read_dataset()
else:
    data = None

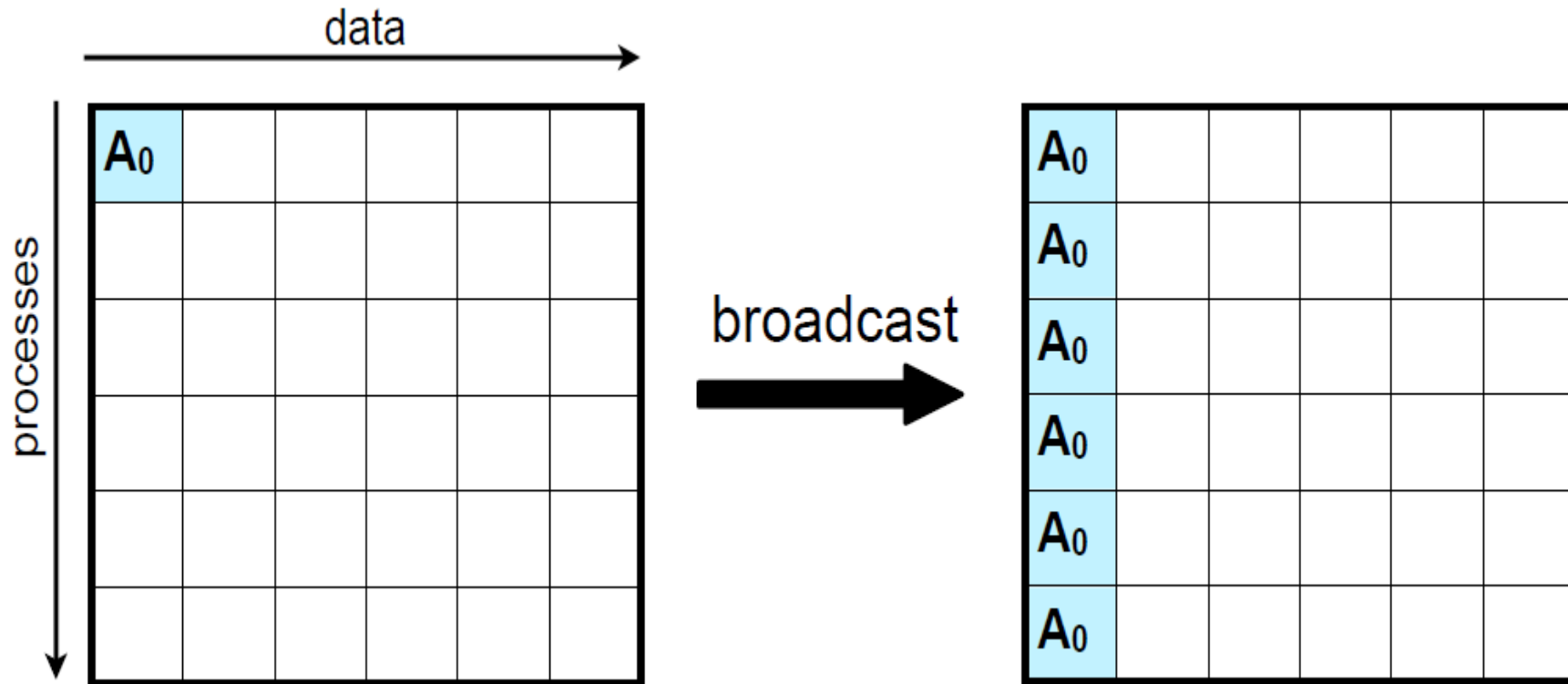
chunk = scatter(data, root=0)
local_result = compute(chunk)
all_results = gather(local_result, root=0)

if rank == 0:
    final = reduce_results(all_results)
```

3. Collective Communication Patterns

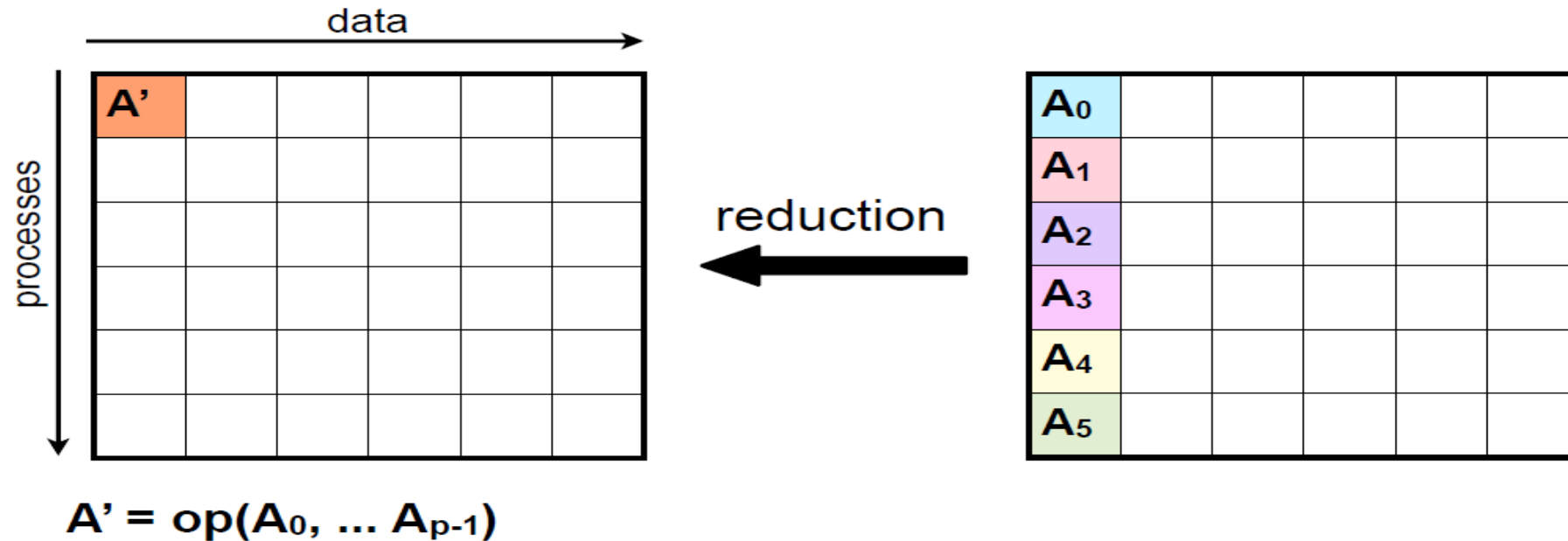
Broadcast, scatter, gather, reduce, all-reduce, barrier

One-to-all Broadcast



AI use: broadcast model parameters, configuration, vocabulary, or random seed

All-to-one Reduction (Reduce)



MPI_Op Predefined Reduction Operations

Operation	Meaning	Datatypes
MPI_MAX	Maximum	integers and floating point
MPI_MIN	Minimum	integers and floating point
MPI_SUM	Sum	integers and floating point
MPI_PROD	Product	integers and floating point
MPI LAND	Logical AND	integers
MPI_BAND	Bit-wise AND	integers and byte
MPI_LOR	Logical OR	integers
MPI BOR	Bit-wise OR	integers and byte
MPI_LXOR	Logical XOR	integers
MPI_BXOR	Bit-wise XOR	integers and byte
MPI_MAXLOC	Max value-location	Data-pairs
MPI_MINLOC	Min value-location	Data-pairs

Reduce and All-Reduce

Combine values such as sum, max, min, average

Σ

Reduce

Many ranks send values; root receives one combined result.

Example: total word count or global accuracy numerator.

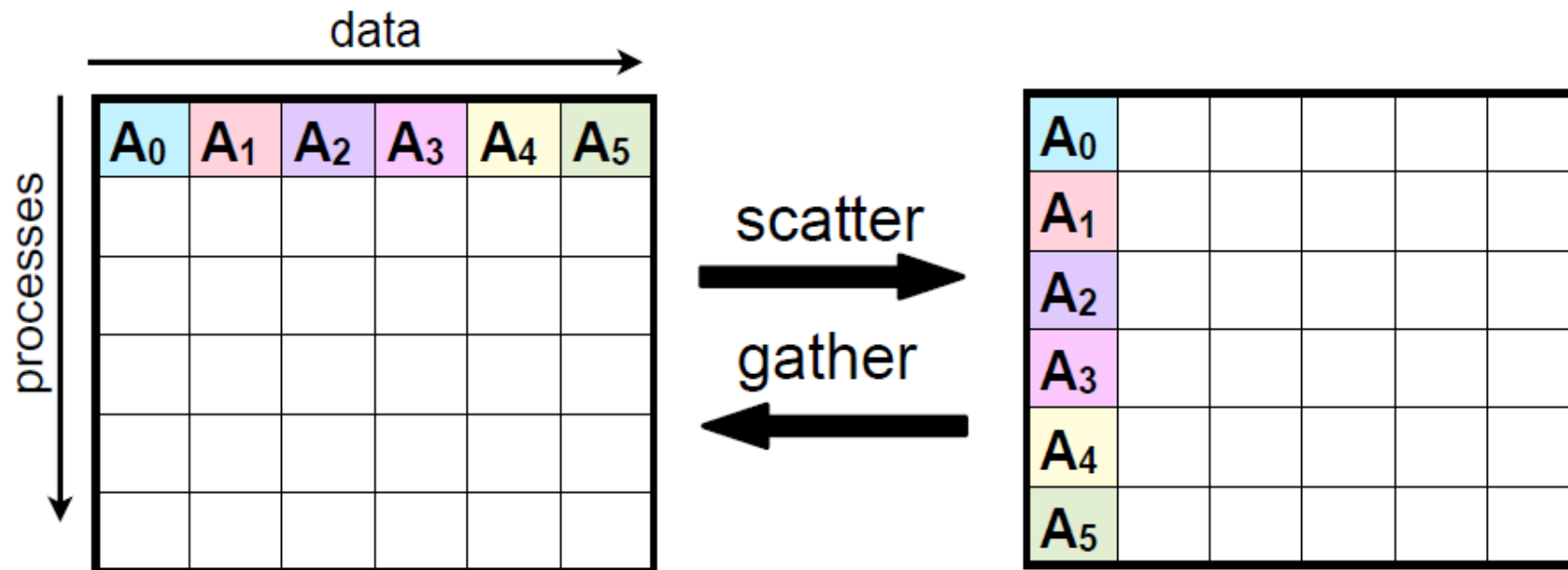
all

All-reduce

Every rank receives the combined result.

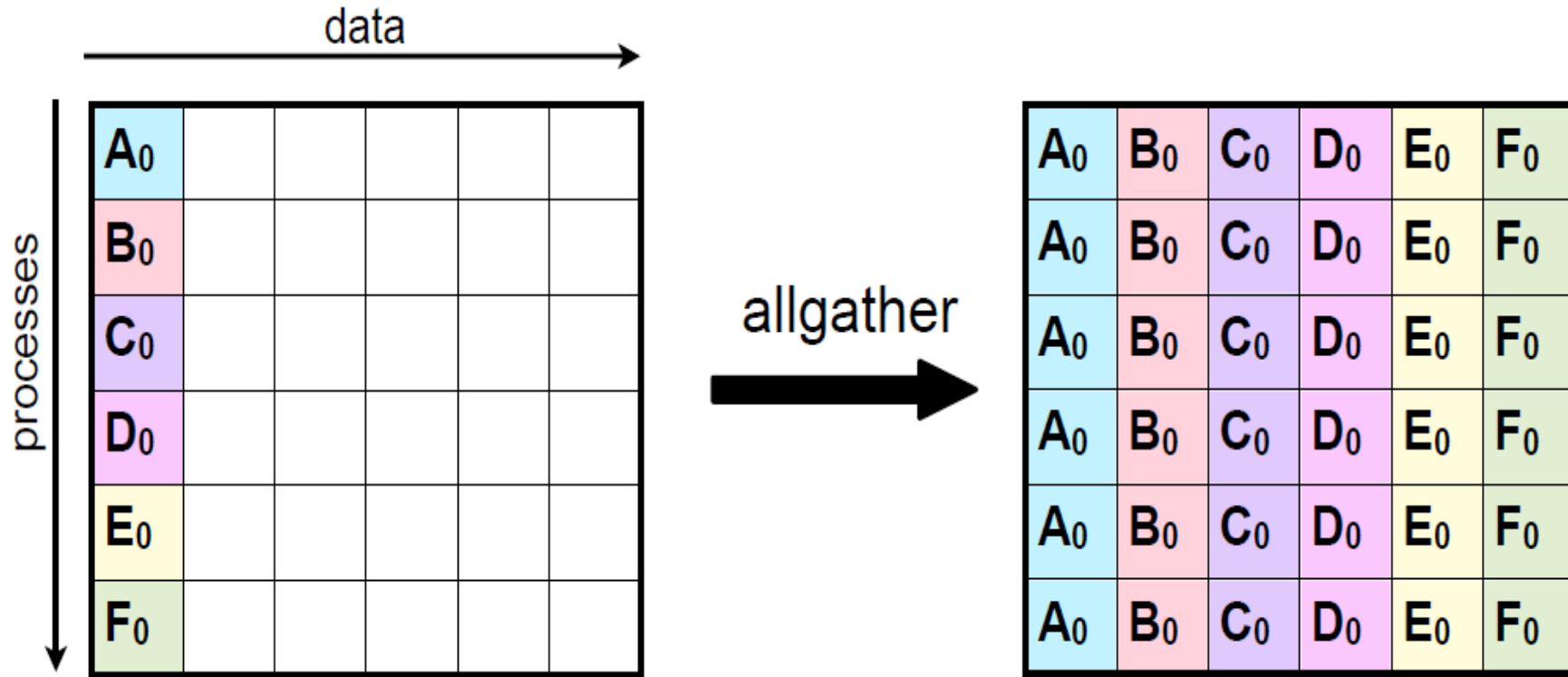
Example: gradient averaging concept for data-parallel training.

Scatter/Gather



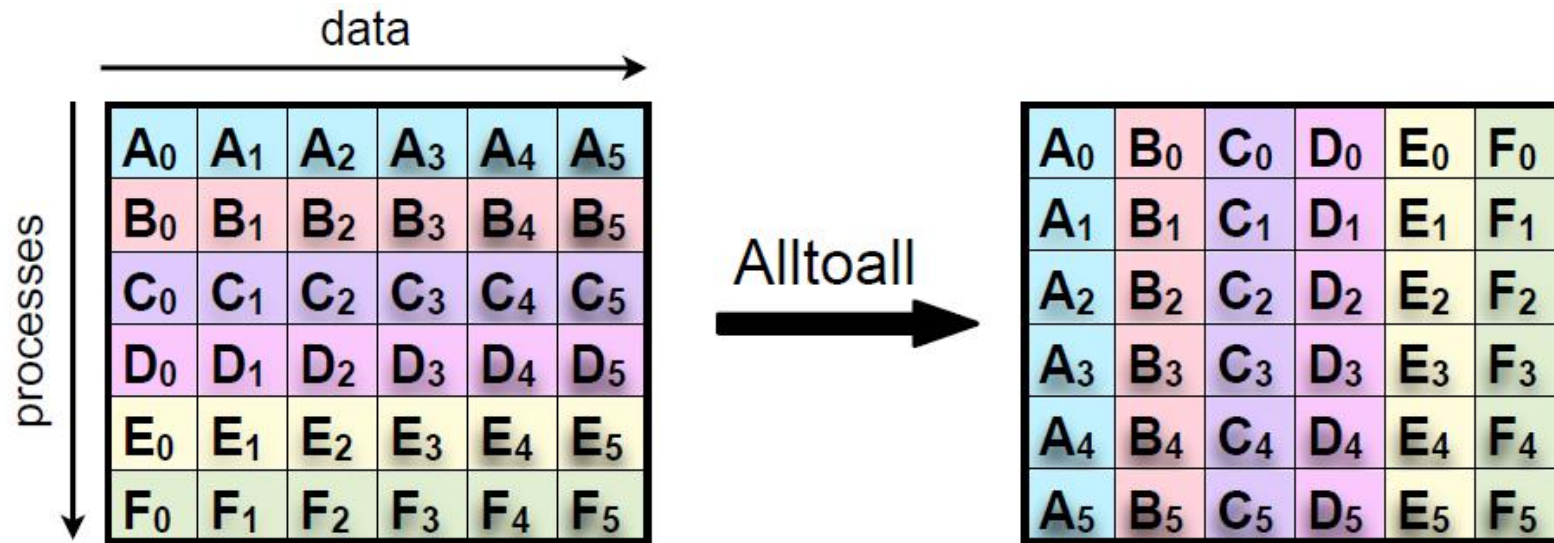
AI use: partition dataset examples, users, documents, images, or prompts

Allgather



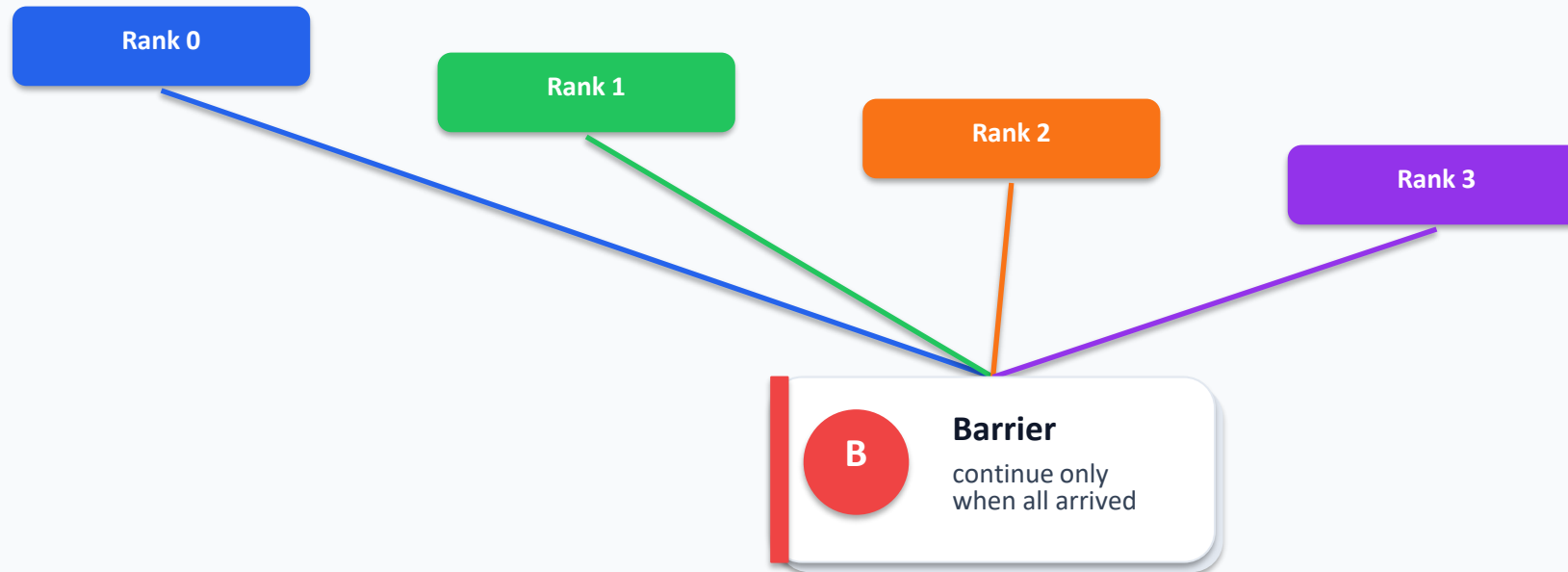
All-to-All Personalized Communication

- Each process starts with its own set of blocks, one destined for each process
- Each process finishes with all blocks destined for itself
- Analogous to a matrix transpose



Barrier: Synchronization Point

All ranks wait until everyone arrives



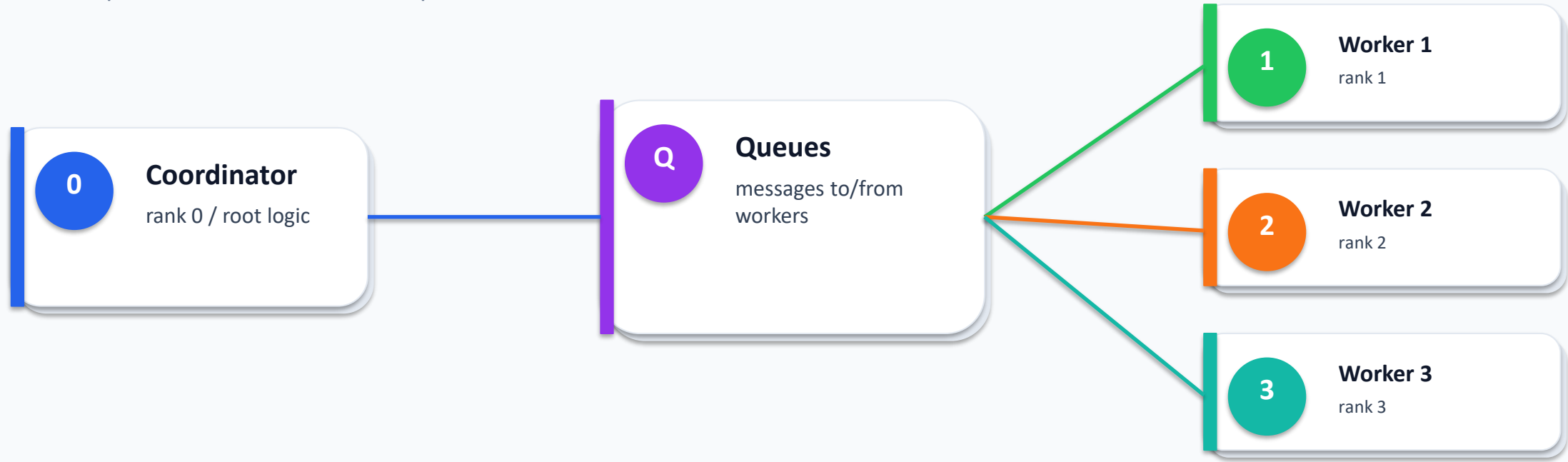
Useful for timing experiments, but unnecessary barriers can slow systems

4. Simulation Design

Building mini-MPI with multiprocessing and queues

How We Simulate MPI Today

One main process coordinates worker processes



Simulation Limitations

We learn the idea, not full MPI performance



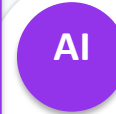
What we simulate

rank IDs,
scatter/gather/reduce
patterns, worker coordination,
correctness checks



What we simplify

network topology, true MPI
runtime, optimized collective
algorithms, fault tolerance



Why it matters

students learn distributed
reasoning before real clusters
or torch.distributed

Deadlocks and Collective Rules

Distributed bugs often mean someone waited forever

1

All ranks participate

missing participant can hang a collective

2

Same order

collectives must be called consistently

3

Matching shapes

send/receive sizes must match expectations

4

Root agreement

everyone must know who the root is

5

Timeout/debug logs

rank-aware logging helps debugging

6

Small tests first

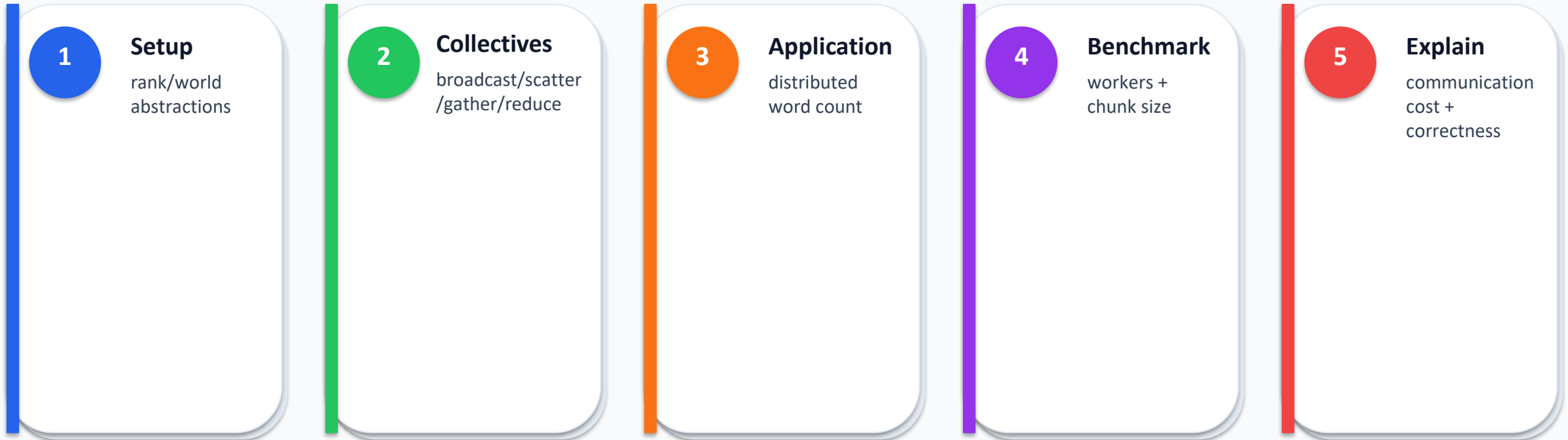
use 2–4 ranks before scaling up

5. Day 5 Lab

Implement mini-MPI and distributed map–reduce

Lab: mini-MPI Collective Simulator

Goal: implement collectives and apply them to word count or data analysis



Deliverable: mini_mpi.py + demo + runtime table + workflow diagram

Starter Code: Chunking and Local Work

Begin with deterministic chunks and local computation



```
from collections import Counter

def chunk_data(items, num_chunks):
    chunks = [[] for _ in range(num_chunks)]
    for i, item in enumerate(items):
        chunks[i % num_chunks].append(item)
    return chunks

def local_word_count(lines):
    c = Counter()
    for line in lines:
        c.update(line.lower().split())
    return c

def reduce_counters(counters):
    total = Counter()
    for c in counters:
        total.update(c)
    return total
```

Starter Code: Serial Baseline

Always verify distributed outputs against serial outputs

```
def make_fake_lines(n=50000):  
    vocab = ["ai", "data", "model", "parallel", "rank",  
            "scatter", "gather", "reduce", "python"]  
    return [" ".join(vocab[(i+j) % len(vocab)] for j in range(20))  
            for i in range(n)]  
  
lines = make_fake_lines()  
serial = reduce_counters([local_word_count(lines)])  
print(serial.most_common(5))
```

mini-MPI: Scatter Function

Root conceptually splits data and assigns chunks to ranks

```
def scatter(data, world_size):  
    #Return one chunk per rank.  
    return chunk_data(data, world_size)
```

```
world_size = 4  
chunks = scatter(lines, world_size)  
for rank, chunk in enumerate(chunks):  
    print(rank, len(chunk))
```

mini-MPI: ProcessPool Gather + Reduce

Simulate ranks with worker processes

```
from concurrent.futures import ProcessPoolExecutor

def worker_count(args):
    rank, chunk = args
    local = local_word_count(chunk)
    return {"rank": rank, "counter": local, "n": len(chunk)}

def distributed_word_count(lines, world_size=4):
    chunks = scatter(lines, world_size)
    tasks = [(rank, chunk) for rank, chunk in enumerate(chunks)]

    with ProcessPoolExecutor(max_workers=world_size) as ex:
        outputs = list(ex.map(worker_count, tasks))

    counters = [out["counter"] for out in outputs]
    return reduce_counters(counters), outputs
```

mini-MPI: Reduce and All-Reduce

Reduce returns one result; all-reduce shares it with everyone

```
def reduce_sum(values):  
    return sum(values)  
  
def all_reduce_sum(values):  
    total = reduce_sum(values)  
    return [total for _ in values]  
  
local_counts = [10, 12, 9, 11]  
print("reduce:", reduce_sum(local_counts))  
print("all_reduce:", all_reduce_sum(local_counts))
```

Step 5: Benchmark and Check Correctness

Compare serial vs simulated distributed workflow

```
import time

for world_size in [1, 2, 4, 8]:
    start = time.perf_counter()
    dist_counter, outputs = distributed_word_count(lines, world_size)
    elapsed = time.perf_counter() - start

    print({
        "world_size": world_size,
        "time": elapsed,
        "lines_per_sec": len(lines) / elapsed,
        "correct": dist_counter == serial,
        "chunk_sizes": [out["n"] for out in outputs],
    })
```

Optional Real MPI Preview: mpi4py Pattern

Same mental model if students later use real MPI



```
# Conceptual mpi4py pattern; not required for this course day.
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
world_size = comm.Get_size()

if rank == 0:
    chunks = scatter(data, world_size)
else:
    chunks = None

chunk = comm.scatter(chunks, root=0)
local_result = compute(chunk)
all_results = comm.gather(local_result, root=0)
```

Mini Challenge

Build, test, and explain mini-MPI

1

Implement

scatter, gather,
reduce_sum,
all_reduce_sum, and
broadcast.

2

Apply

distributed word count or
numeric statistics over
chunks.

3

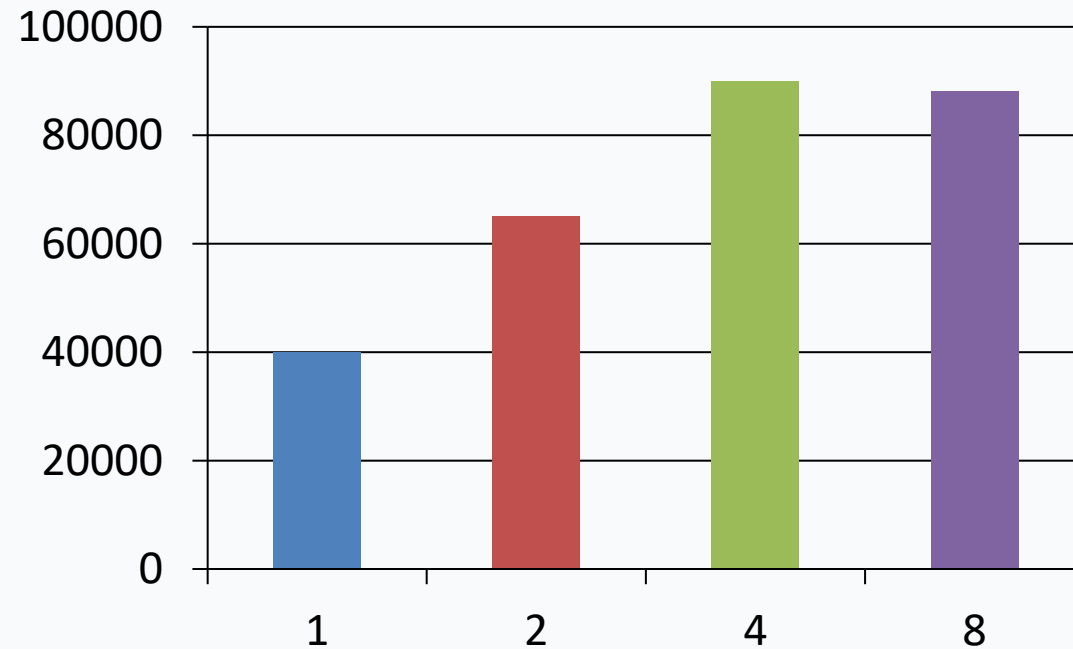
Explain

communication pattern,
correctness, speedup, and
bottlenecks.

Expected Result Format

Students should report correctness and communication cost

Example distributed word-count throughput



Correctness

distributed result equals serial baseline



Load balance

chunk sizes and runtimes similar



Communication

scatter/gather/reduce cost explained

Exit Ticket

Short reflection + mini-MPI demo

Q

Answer

- What is rank and world size?
- Scatter vs broadcast?
- Gather vs reduce?
- What caused speedup to flatten?
- Where will your project use collectives?

✓

Submit

- mini_mpi.py
- demo script or notebook
- runtime/correctness table
- architecture diagram
- 1-paragraph explanation

Instructor Notes

Teaching moves for Day 5

- 1 Emphasize patterns, not real MPI installation complexity.
- 2 Use rank/world size language consistently all day.
- 3 Require serial baselines for correctness.
- 4 Make students draw the communication pattern before coding.
- 5 Discuss deadlocks and collective participation rules.
- 6 Connect collectives to Day 6 gradient averaging.

References and Further Reading

MPI collectives and distributed communication

1. **MPI Tutorial: Scatter sends array chunks to processes, while Gather collects results back to a root process.**
2. **LLNL HPC Tutorials: collective operations include synchronization, data movement such as broadcast/scatter/gather, and reductions.**
3. **MPI standard overview: collective communication includes barrier, broadcast, gather, scatter, allgather, reduce, and allreduce operations.**
4. **Optional mpi4py learning resources: broadcast, scatter, gather, and reduce patterns in Python.**